



Southeast University

PROJECT REPORT

Project Name: LIBRARY MANAGEMENT SYSTEM

Submitted By: Md. Abdul Latif Khan Tanvir
ID: 2025000000204

Group Members: Md. Abdul Latif Khan Tanvir
(2025000000204)
Mst. Shohagi Akter
(2025000000202)
Mir Mohammad Fahim Hossain
(2025000000200)

Submitted To: Md. Ayon Mia
Lecturer at
Southeast University

Course Code: CSE282.6

Course Title: Introduction to Programming Language II (Java) Lab

Date of Submission: 28 September 2026

Table of Contents

<u>1. Introduction.....</u>	<u>3</u>
<u>2. Objectives.....</u>	<u>3</u>
<u>3. Motivation.....</u>	<u>4</u>
<u>4. Applications.....</u>	<u>4</u>
<u>5. Project Description.....</u>	<u>4-6</u>
<u>6. Tools & Technologies.....</u>	<u>6</u>
<u>7. Output / Results.....</u>	<u>7-13</u>
<u>8. Contribution.....</u>	<u>14</u>
<u>9. Conclusion.....</u>	<u>14-15</u>
<u>10. GitHub Link.....</u>	<u>15</u>

1. Introduction

The Library Management System is a Java-based desktop application developed to simplify and organize the management of books and library members. The system provides a user-friendly graphical interface through which users can add, view, update, and delete book and member information.

The application is developed using Java Swing, Object-Oriented Programming (OOP) principles, and file handling for data storage. It also includes input validation, exception handling, and report generation features. The system stores book and member information in text files, allowing the data to remain available after the application is closed and reopened.

Overall, the project demonstrates the practical implementation of Java programming concepts in developing a simple and functional Library Management System.

2. Objectives

What the project aims to achieve:

The project aims to develop a simple, user-friendly, and functional Library Management System that can efficiently manage library books and members through a graphical user interface. It is designed to reduce the complexity of maintaining library records manually and provide an organized way to perform common library management activities.

Specific goals of the project:

The project aims to apply fundamental Java programming and Object-Oriented Programming (OOP) concepts in a practical software project. It focuses on implementing encapsulation, abstraction, inheritance, and polymorphism, along with file handling, input validation, exception handling, and persistent data storage. Another goal is to provide proper error and success messages so that users can interact with the system easily and correctly.

Key functionalities to be implemented:

The system implements Create, Read, Update, and Delete (CRUD) operations for books and library members. It allows users to add and view book and member records, update existing member information, and delete member records. The system also provides input validation, prevents duplicate member IDs, handles invalid data through custom exception handling, stores records using text files, and generates a library report containing book and member information with their total counts.

3. Motivation

Why we selected/developed this project:

We selected this project to develop a practical application that demonstrates the use of Java programming and Object-Oriented Programming (OOP) concepts in a real-world system. A Library Management System provides an appropriate environment to apply concepts such as encapsulation, abstraction, inheritance, and polymorphism while also working with GUI development, file handling, CRUD operations, and exception handling.

Problem or real-world need that motivated it:

Managing books and member information manually can make it difficult to maintain, update, and retrieve library records efficiently. This project was developed to provide a simple computerized system where book and member information can be stored, viewed, updated, and deleted in an organized manner. The system also provides validation, error handling, and report generation to make common library management tasks more structured and convenient.

4. Applications

Where the project can be used:

The Library Management System can be used in schools, colleges, universities, and small libraries where books and member records need to be managed in an organized manner.

Potential users of the system:

The potential users include librarians, library staff, and authorized users responsible for managing books and library member information.

Practical uses and benefits:

The system can be used to add, view, update, and delete book and member records, maintain data through file-based storage, validate user input, handle errors, and generate library reports. It helps organize library records and makes common management tasks easier and more systematic.

5. Project Description

Overall description of the system diagram:

The Library Management System is organized into several components that work together to provide the complete functionality of the application. The **Main** class starts the application and opens the **MainFrame**, which acts as the main GUI window. The GUI communicates with the system's model and file-handling components to manage books, members, and reports. The **FileManager** handles the storage and modification of data in **books.txt** and **members.txt**, while the generated library report is stored in **library_report.txt**.

System Flow:

Main Class (Main.java)



Main GUI (MainFrame.java)



Book Management

- Add Book
- View Books
- Update Book
- Delete Book

Member Management

- Add Member
- View Members
- Update Member
- Delete Member

Report Generation

- Generate Library Report



File Management (FileManager.java)



Data Storage

- books.txt
- members.txt
- library_report.txt

Main features and functionalities:

The system provides book and member management functionalities through a graphical user interface. The main features include adding, viewing, updating, and deleting records, input validation, duplicate Member ID prevention, custom exception handling, file-based data persistence, error and success messages, and library report generation. The system also allows users to view the total number of books and members through the generated report.

Major modules/classes of the system:

The major classes of the system are organized into different components. The **Main** class serves as the entry point of the application. The **model** package contains **Person**, **Member**, and **Book**. The **file** package contains **FileManager** for file operations, while the **exception** package contains **InvalidDataException** for handling invalid data. The **gui** package contains **MainFrame**, **AddBookPanel**, **ViewBookPanel**, **AddMemberPanel**, **ViewMemberPanel**, **UpdateMemberPanel**, **DeleteMemberPanel**, and **ReportPanel** for the graphical user interface.

Workflow of the system:

The workflow begins when the user runs the application through the **Main** class. The **MainFrame** then displays the available library management options. The user can select the required operation, such as adding or viewing books and members, updating or deleting member records, or generating a report. The system validates the entered information and performs the requested operation through the appropriate GUI and file-handling components. The updated information is stored in the corresponding text files, and the generated report is saved in **library_report.txt**.

Java/OOP concepts used in the project:

The project applies the major Object-Oriented Programming concepts of Encapsulation, Abstraction, Inheritance, and Polymorphism. Encapsulation is implemented through private fields and getter/setter methods in the model classes. Abstraction is implemented through the abstract **Person** class. Inheritance is demonstrated by the **Member** class extending **Person**. Polymorphism is implemented through the overriding of the **displayInfo()** method in the **Member** class. The project also uses Java Swing for GUI development, file handling for persistent data storage, and exception handling for managing invalid input and file-related errors.

6. Tools & Technologies

Programming Language and Version:

Java (JDK 17)

IDE Used for Development:

IntelliJ IDEA

Libraries/Frameworks Used:

Java Swing for graphical user interface development and Java I/O libraries for file handling and data storage.

7. Output / Results

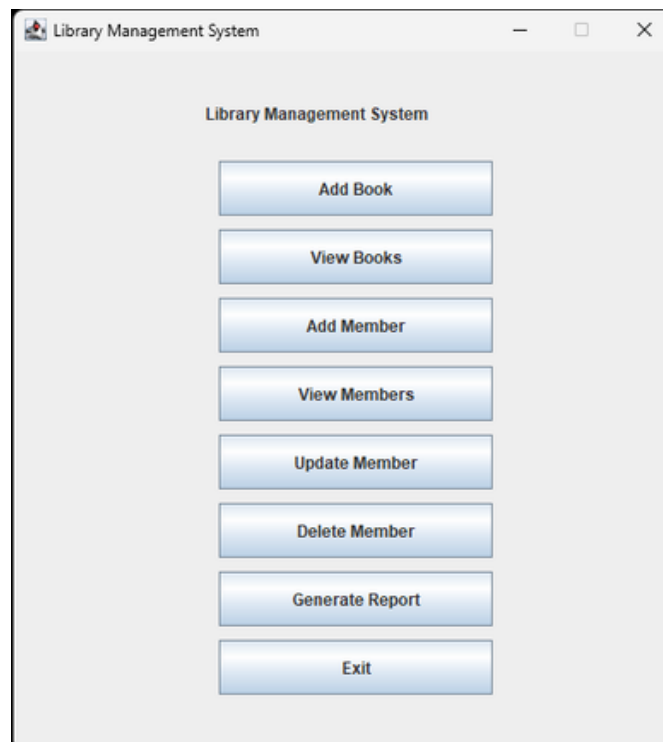


Figure 7.1: Home Interface of the Library Management System

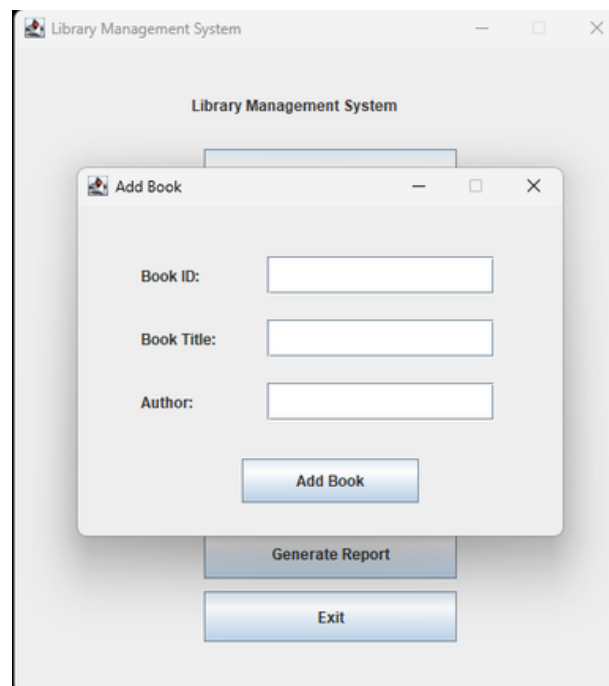


Figure 7.2: Add Book Interface

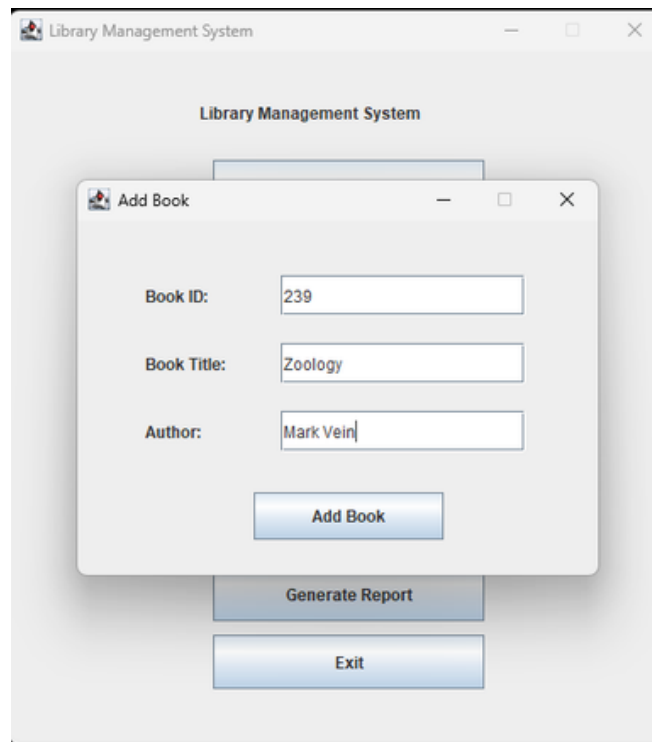


Figure 7.3: Add Book Interface with Sample Input

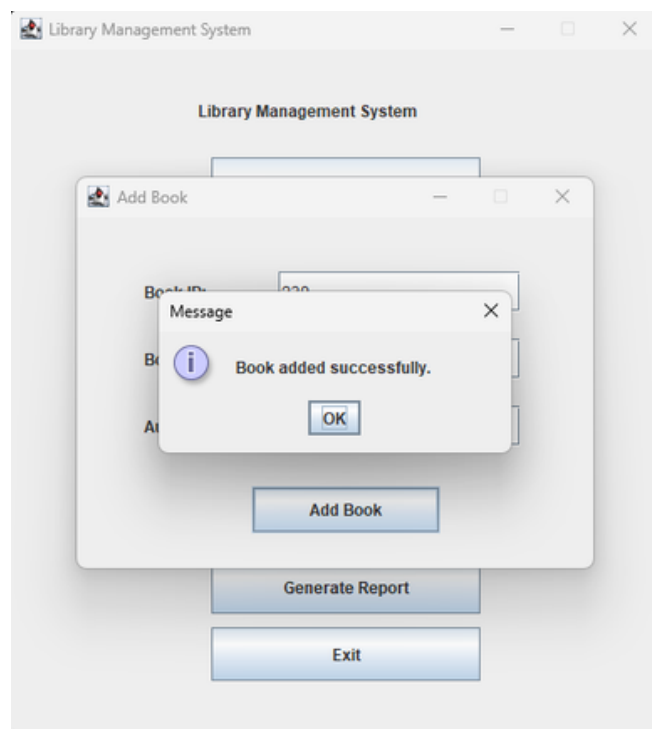


Figure 7.4: Successful Book Addition Message

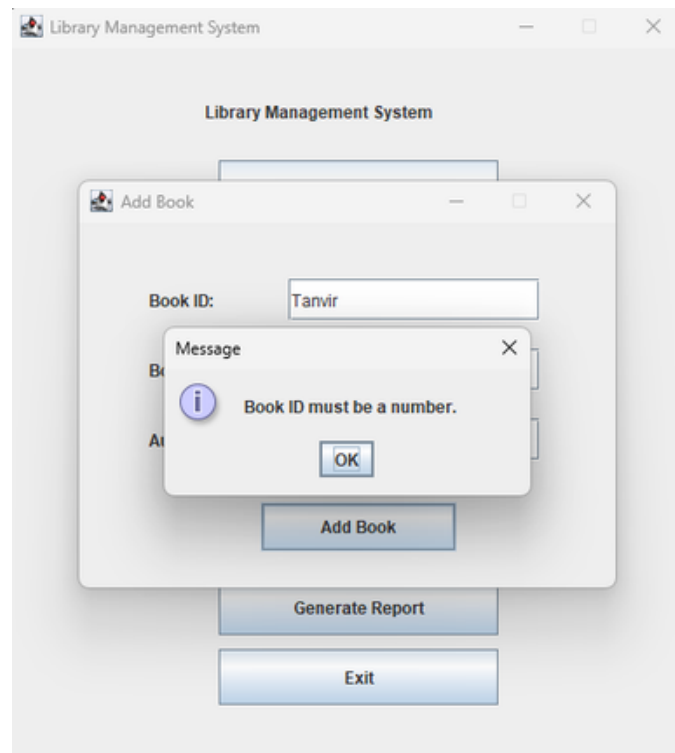


Figure 7.5: Book ID Validation Error Message

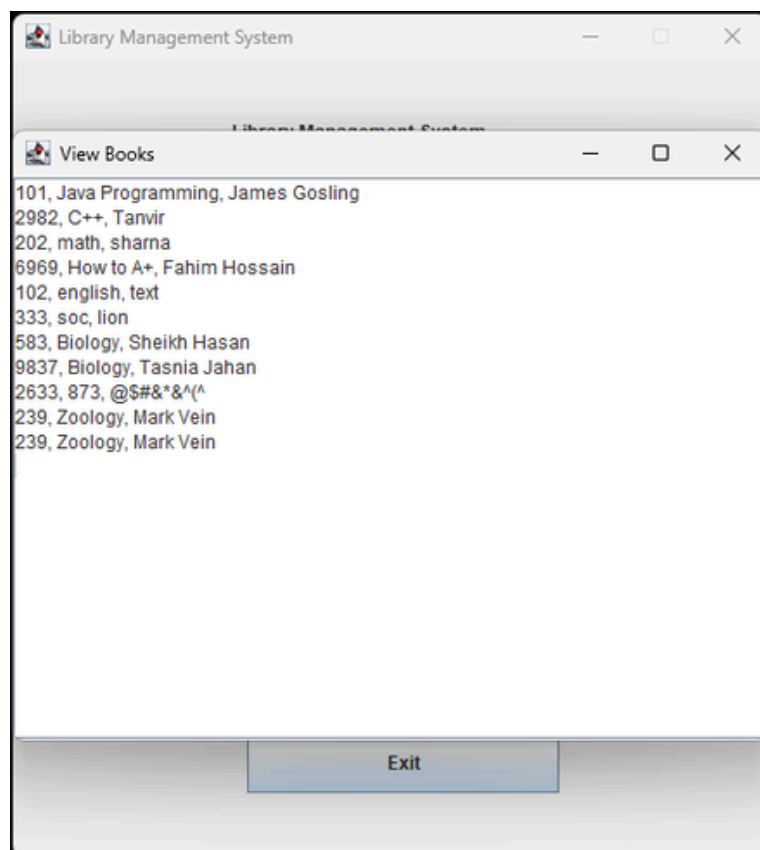


Figure 7.6: Display of Available Books in View Books

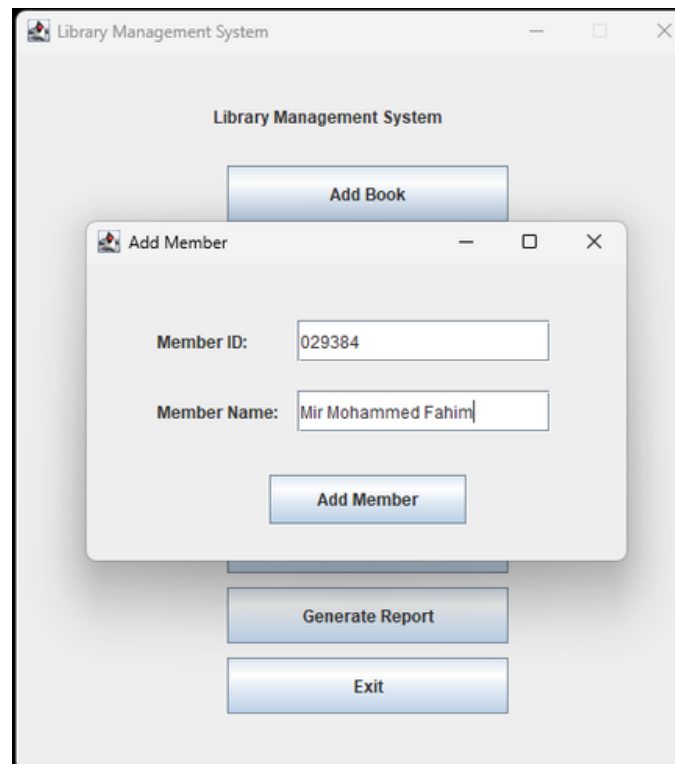


Figure 7.7: Add Member Interface with Sample Input

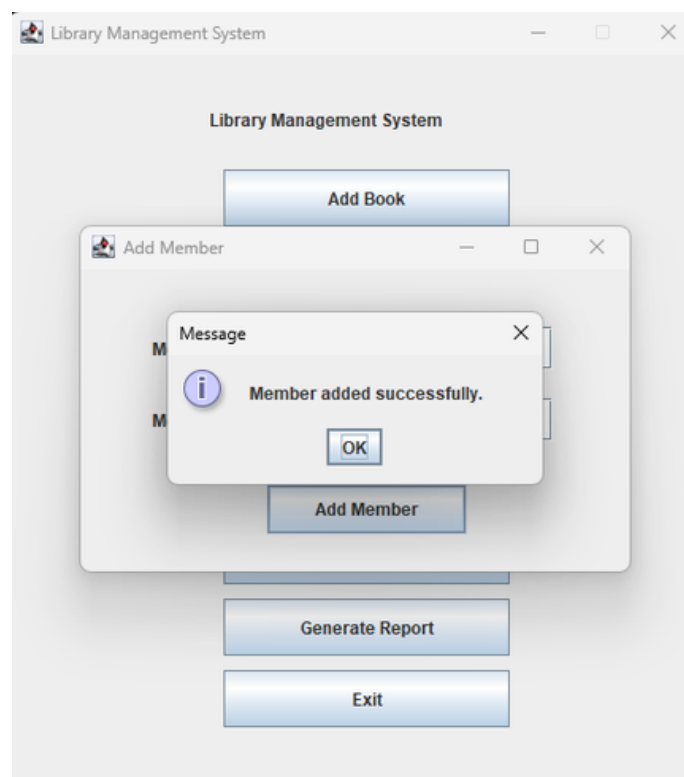


Figure 7.8: Successful Member Addition Message

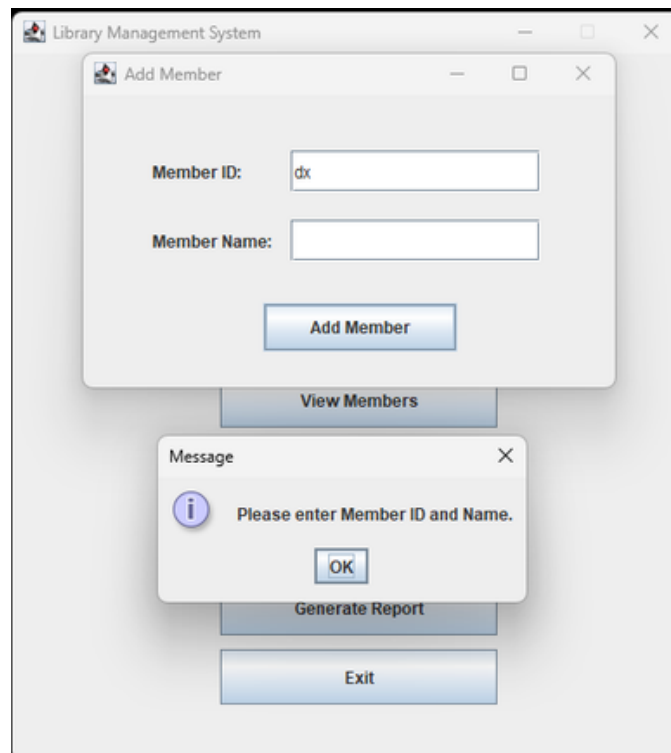


Figure 7.9: Add Member Input and Validation Error Message

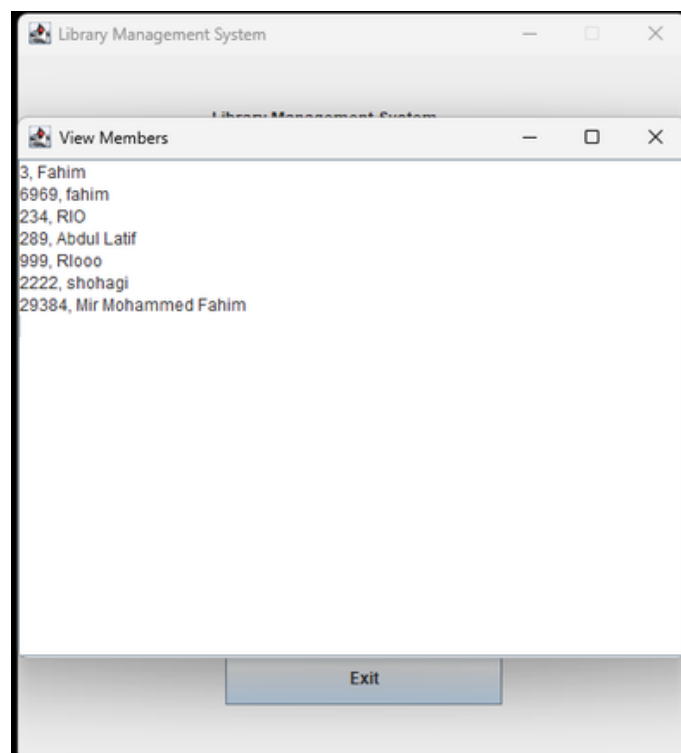


Figure 7.10: Display of Registered Members

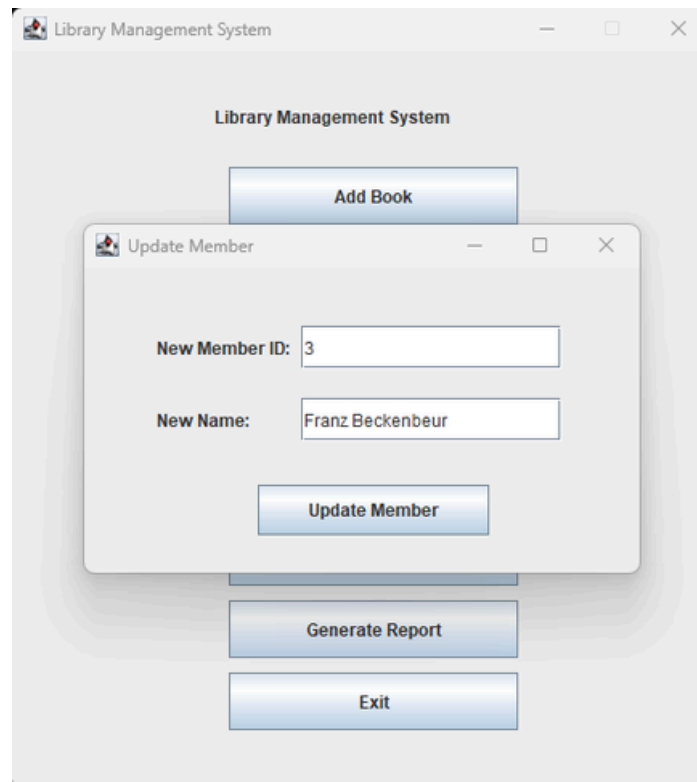


Figure 7.11: Update Member interface and Member input

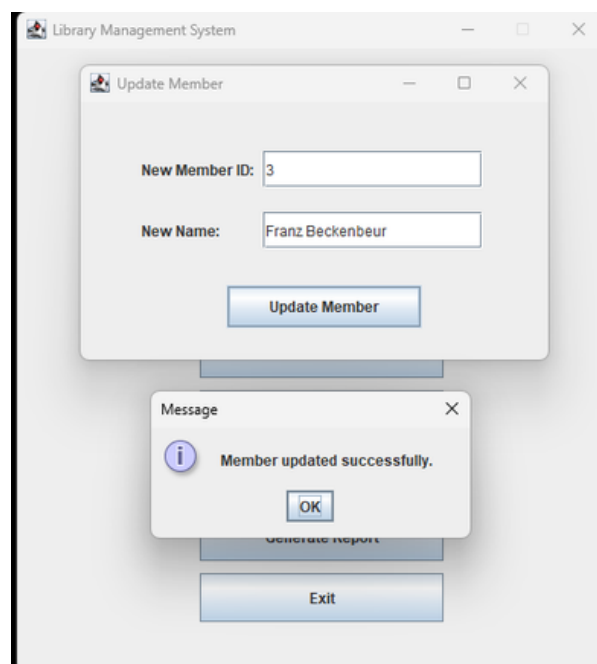


Figure 7.12: Successful Member Update Message

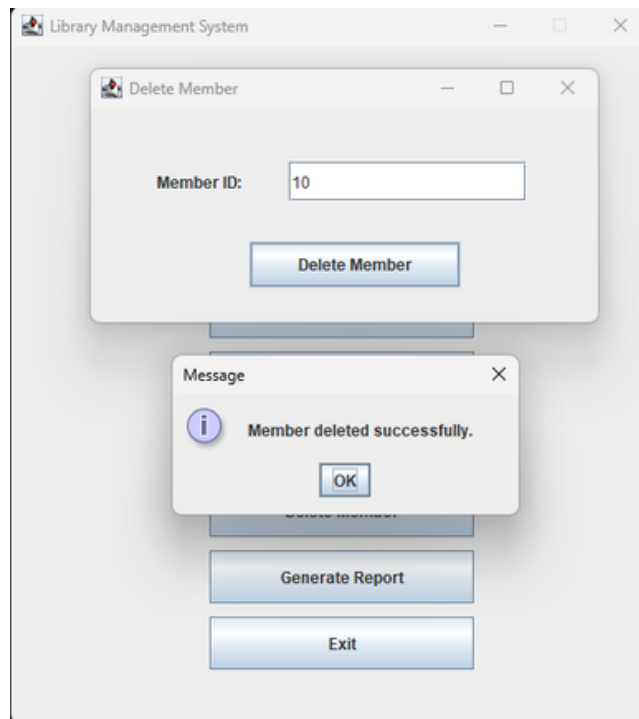


Figure 7.13: Delete Member Interface and input and successful Member Deletion Message

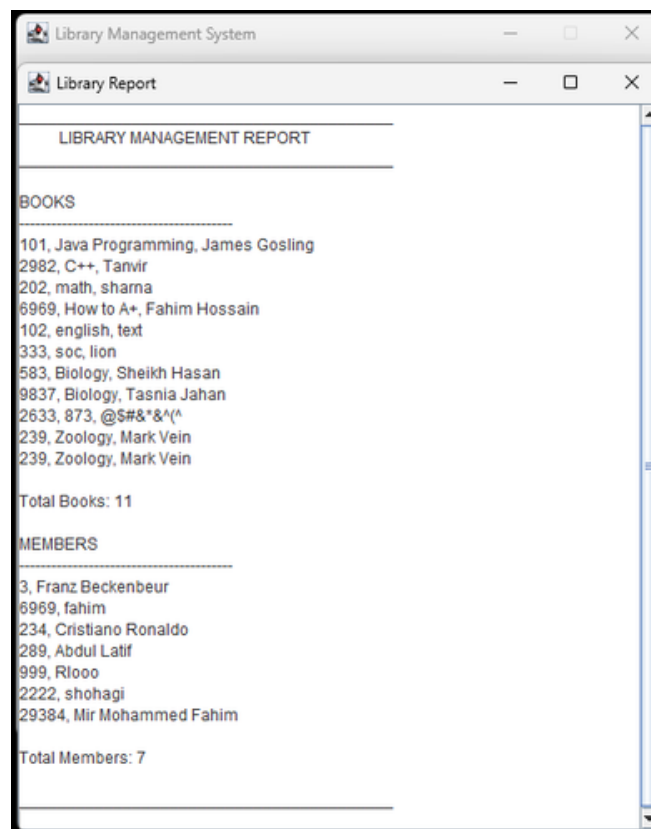


Figure 7.14: Generated Library Management Report

8. Contribution

Group Members	Individual Contribution	Tasks Completed	Modules / Features Developed
M. A. Latif Khan Tanvir (2025000000204)	File handling, GUI, book management, debugging and integration	Project structure, file handling, Book Update/Delete, GUI integration, validation, Member Read, bug fixing, GUI improvement, code cleanup, README	File Handling, Book CRUD, Member Read, Validation, GUI, Debugging, README
Mst. Shohagi Akter (2025000000202)	OOP, book management, reporting and GUI styling	OOP classes, Book CRUD, View Books, Encapsulation, Report Generation, button styling	OOP Model, Book CRUD, Report, Encapsulation, GUI Styling
Mir Mohammad Fahim Hossain (2025000000200)	Member management and exception handling	Member CRUD, Add Member, custom exceptions, validation, GUI model	Member CRUD, Validation, Exception Handling, GUI Model

9. Conclusion

Summary of the completed project:

The Library Management System was successfully developed as a Java-based desktop application using Java Swing, Object-Oriented Programming, file handling, and exception handling. The system provides book and member management, CRUD operations, input validation, persistent file-based data storage, and library report generation through a graphical user interface.

Whether the objectives were achieved:

The major objectives of the project were achieved. The system successfully implements the required book and member management functionalities and demonstrates the major OOP concepts of Encapsulation, Abstraction, Inheritance, and Polymorphism. The completed system was also tested across its main functionalities, including validation, CRUD operations, GUI interaction, and report generation.

Limitations of the project:

The current system uses text files for data storage instead of a database. It is designed as a simple desktop application and does not include advanced features such as user authentication, multiple user roles, online/cloud data access, book borrowing/return tracking, or automated database backup.

Possible future improvements:

The system can be further improved by integrating a database management system, adding user authentication and role-based access, implementing book borrowing and return functionality, adding search and filtering options, improving the GUI design, and providing more advanced reports and data management features.

GitHub Repository Link:

https://github.com/TanvirKhanio/Library_Management_System